Universidade Estadual da Paraíba

ES6

PROFESSOR:

Allan Vilar de Carvalho

allanvilarcarvalho@gmail.com

UEPB
Universidade
Estadual da Paraíba

Tópicos da Aula

- Atribuição de desestruturação
- Modelo Literal
- Declarações
- Sintaxe de classe
- Separar script em módulos
- Promessa

ES6

- Atribuição de desestruturação: para passar um objeto como parâmetro de uma função

```
const profileUpdate = (profileData) => {
  const { name, age, nationality, location } = profileData;

}
```

- Ex.:

```
const profileUpdate = ({ name, age, nationality, location }) => {

}
```

ES6

- Modelo Literal
  - Permite: criar strings complexas

  - Crase
  - Multilinha: não necessita de \n
  - ${variable}
    - ${a + b}

  - Ex.:

```
const person = {
  name: "Zodiac Hasbro",
  age: 56
};

const greeting = `Hello, my name is ${person.name}!
I am ${person.age} years old.`;

console.log(greeting);
```

ES6

- Declarações literais de objetos
  - Usa: abreviação de propriedades de objetos

```
const getMousePosition = (x, y) => ({
  x: x,
  y: y
});
```

  - Ex.:

```
const getMousePosition = (x, y) => ({ x, y });
```

ES6

- Declarações de funções dentro de objetos
  - Remoção: palavra function; dois pontos

```
const person = {
  name: "Taylor",
  sayHello: function() {
    return `Hello! My name is ${this.name}.`;
  }
};
```

  - Ex.:

```
const person = {
  name: "Taylor",
  sayHello() {
    return `Hello! My name is ${this.name}.`;
  }
};
```

ES6

- Sintaxe de classe
  - palavra-chave: class
  - UpperCamelCase

```
// Explicit constructor
class SpaceShuttle {
  constructor(targetPlanet) {
    this.targetPlanet = targetPlanet;
  }
  takeOff() {
    console.log("To " + this.targetPlanet + "!");
  }
}
const zeus = new SpaceShuttle('Jupiter');
// prints To Jupiter! in console
zeus.takeOff();
```

```
// Implicit constructor
class Rocket {
  launch() {
    console.log("To the moon!");
  }
}
const atlas = new Rocket();
// prints To the moon! in console
atlas.launch();
```

ES6

- Sintaxe de classe
  - getters e setters
    - controla o acesso
    - obtém/define valores

```
class Book {
  constructor(author) {
    this._author = author;
  }
  // getter
  get writer() {
    return this._author;
  }
  // setter
  set writer(updatedAuthor) {
    this._author = updatedAuthor;
  }
}
const novel = new Book('anonymous');
console.log(novel.writer);
novel.writer = 'newAuthor';
console.log(novel.writer);
```

ES6

- Separar script em módulos
  - Permite: compartilhar códigos entre arquivos JavaScript
    - Exportar/Importar
  - Primeiro:

```
<script type="module" src="filename.js"></script>
```

  - Segundo: Exportar blocos de código

```
export const add = (x, y) => {
  return x + y;
}
```

```
const add = (x, y) => {
  return x + y;
}

export { add };
```

```
export { add, subtract };
```

ES6

- Separar script em módulos
  - Permite: compartilhar códigos entre arquivos JavaScript
    - Exportar/Importar
  - Terceiro: Importar blocos de código

```
import { add } from './math_functions.js';
```

```
import { add, subtract } from './math_functions.js';
```

ES6

- Separar script em módulos
  - Permite: compartilhar códigos entre arquivos JavaScript
    - Exportar/Importar

  - Importar tudo
    - Ex.: cria um objeto chamado myMathModule

```
import * as myMathModule from "./math_functions.js";
```

```
myMathModule.add(2,3);
myMathModule.subtract(5,3);
```

ES6

- Separar script em módulos
  - Permite: compartilhar códigos entre arquivos JavaScript
    - Exportar/Importar

  - Exportar apenas um valor: export default

```
export default function add(x, y) {
  return x + y;
}
```

ES6

- Separar script em módulos
  - Permite: compartilhar códigos entre arquivos JavaScript
    - Exportar/Importar

  - Importar um export default

```
import add from "./math_functions.js";
```

ES6

- Promessa
  - promessa de que vai fazer uma tarefa

  - Criação
    - new Promise
    - Parâmetros: *resolve* e *reject*

```
const myPromise = new Promise((resolve, reject) => {


});
```

ES6

- Promessa
  - promessa de que vai fazer uma tarefa
  - Conclusão
    - *resolve/reject*: método; recebe apenas um argumento

```
const myPromise = new Promise((resolve, reject) => {
  if(condition here) {
    resolve("Promise was fulfilled");
  } else {
    reject("Promise was rejected");
  }
});
```

ES6

- Promessa
  - promessa de que vai fazer uma tarefa

  - Manipulação
    - após *resolve*
      - método then
      - result : argumento dado ao método resolve

```
myPromise.then(result => {


});
```

ES6

- Promessa
  - promessa de que vai fazer uma tarefa

  - Manipulação
    - após *reject*
      - método catch
      - error: argumento dado ao método reject

```
myPromise.catch(error => {


});
```

Dúvidas?